

Cryptography, Theory and Practice CIS 4930/6371 (Fall 2026)

Instructor: Dr. Attila A. Yavuz

TA: Sean Hernandez

Contributor: Sheng Rao

September 11, 2026

Homework 2 (**Due: Sep. 21, 2026 – 23 : 59**)[**100 Points**]

(Upload your solutions to the CANVAS)

Requirements: HW groups are 2 people. Each student should contribute all parts of the answers. If one group member drops, the other is responsible (each HW can be handled by a single student if needed, so no excuse for missing a partner. See syllabus for breaking-up with a programming partner). Directly borrowing (e.g., copy-paste) from any material and putting it in your solutions (e.g, from online solutions, Wikipedia, or research papers) is plagiarism (see Syllabus for its corresponding actions). Please cite very carefully each resource you use but citing a solution does not give a license to directly put it as an answer. **All of your answers must be in your own words and interpretations.**

Note: Many of these questions require you to recall explanations/discussions during the class (this is why attendance is important), and also do research on the internet to find good answers. Please kindly note that very brief and/or informal explanations will potentially not be sufficient.

Remark: Answers to questions should be prepared by a text editor (e.g., Microsoft Word or Latex). Handwritten submissions are not accepted. In your answer key, please indicate the point of each question as it is done in this HW document. Preferably, take questions from here, plug them into your answer key with their sub scores, and provide your answer below them. This will facilitate the grading.

As indicated in Syllabus, **no partial credit is possible for coding answers that does not produce correct results with respect to test vectors, deviate from the description and rubric, are coded on platforms other than specified, or do not work on the grader's machine.**

1 Programming Assignment 1: Greatest Common Divisor [**10 pt**]

In this exercise we will implement a simple Greatest Common Divisor finder that uses Euclid's algorithm as described in "Crash.NumberTheoryIntro.pptx", slide 7.

GCD:

1. The program reads two numbers directly from the commandline in decimal string format (in other words, they are `argv[1]` and `argv[2]` in your C program).
2. The program calculates the greatest common divisor of the two numbers using Euclid's algorithm.
3. The program prints the greatest common divisor as a string using `printf()`

Programming Assignment 1 - Specific Notes

- **Do not use library functions that calculate the GCD directly.**
You must perform the steps of Euclid's algorithm.
- Since we are sometimes working with numbers larger than 32-bits (but smaller than 64-bits), it is recommended to utilize the built-in `int64_t` data type.
- It is recommended to encapsulate your algorithm logic into a function so that you can easily copy and reuse it in later assignments.
- Compile your code with the following commands:
`gcc gcd.c -o gcd`
- Run your code with the following commands:
`./gcd (firstnumber) (second number)`
For example, "`./gcd 20 15`" should output "5" and nothing else.
- We have provided a script (`VerifyingGCD.sh`) that you can use to test the correctness of your final code. Please, only submit your code if it is functional.
- In order to use the script, just put "`gcd.c`" along with the "`VerifyingGCD.sh`" script in one folder and run the following command in the terminal:
`bash VerifyingGCD.sh`
- The verification script is the same one that is run to grade your submissions, only the test vectors are different.
- **You Must submit 1 file for this part of the assignment:**
`gcd.c`

Rubric:

- Passes the grader's verification **[10 pt]**

2 Programming Assignment 2: Exponentiation through Square-and-Multiply with 256-bit GMP integers **[20 pt]**

In this exercise we will put into action the square-and-multiply algorithm of quickly calculating modular exponentiation as described in "Crash.NumberTheoryIntro.pptx", slide 16. Recall how calculating something like x^n can be simplified into $x(x^2)^{(n-1)/2}$ if n is odd, and $(x^2)^{n/2}$ if n is even, which drastically reduces the amount of multiplications you need to make. We will be using the GMP library to work with large 256-bit integers.

Square-And-Multiply:

1. The program reads the base, the exponent, and the modulo of the exponentiation to be calculated from the files "**Base.txt**", "**Exponent.txt**", and "**Modulo.txt**" respectively.
(Read them as unsigned char so that you can use the functions provided in in-class exercises).
2. The program preforms the square-and-multiply algorithm with the provided numbers using GMP library functions.
3. The program writes the result in decimal string format to a file named "**Result.txt**".

Programming Assignment 2 - Specific Notes

- **Do not use library functions that calculate the modular exponentiation directly.**
You must perform the steps of the Square-and-Multiply algorithm.
- Please refer to the GMP manual (<https://gmplib.org/manual/>) for GMP library functions and how to use them.
- You can assign strings into GMP large integer data type (mpz_t) with the following function:
mpz_set_str(mpz_t output, const char* input, int base)
where you can convert unsigned char from input into output with mpz_t type and since we are using decimal strings, the base is 10.
- It is recommended to encapsulate your algorithm logic into a function so that you can easily copy and reuse it in later assignments.
- Compile your code with the following commands:
gcc snm_gmp.c -lgmp -o snm_gmp
- Run your code with the first test files with the following command:
./snm_gmp Base1.txt Exponent1.txt Modulo1.txt
- We have provided a script (VerifyingSNM.sh) and test vectors that you can use to test the correctness of your final code. Please, only submit your code if it is functional.
- In order to use the script, just put “snm_gmp.c”, the provided test vectors, and the “VerifyingSNM.sh” script in one folder and run the following command in the terminal:
bash VerifyingSNM.sh
- The verification script uses the same format as the one that is run to grade your submissions, with different test vectors.
- **You Must submit 1 file for this part of the assignment:**
snm_gmp.c

Rubric:

- The “Result.txt” file **[20 pt]**

3 Programming Assignment 3: Extended Euclid’s Algorithm with 256-bit GMP integers **[20 pt]**

In this exercise we will put into action using the Extended Euclidean Algorithm to quickly calculate the modular inverse of a large number modulo another large number, as described in “Crash_NumberTheoryIntro.pptx”, slide 10. Recall that the Extended Euclidean Algorithm can be used to calculate the greatest common divisor of two numbers, but also gives us the Bézout coefficients (x and y such that $ax + by = \gcd(a, b)$) that can be used to calculate the modular multiplicative inverse of the provided numbers. We will be using the GMP library to work with large 256-bit integers.

Extended Euclidean Algorithm:

1. The program reads the number and the modulo from the files “Number.txt” and “Modulo.txt” respectively. (Read them as unsigned char so that you can use the functions provided in in-class exercises).
2. The program performs the Extended Euclidean Algorithm with the provided numbers using GMP library functions.

3. The program writes the result in decimal string format to a file named “**Result.txt**”.

Programming Assignment 3 - Specific Notes

- Do not use library functions that calculate the modular inverse or Extended Euclidean Algorithm coefficients directly.
You must perform the steps of the Extended Euclidean Algorithm yourself.
- Please refer to the GMP manual (<https://gmplib.org/manual/>) for GMP library functions and how to use them.
- It is recommended to encapsulate your algorithm logic into a function so that you can easily copy and reuse it in later assignments.
- Compile your code with the following commands:
`gcc eea_gmp.c -lgmp -o eea_gmp`
- Run your code with the first test files with the following command:
`./eea_gmp Number1.txt Modulo1.txt`
- We have provided a script (VerifyingSNM.sh) and test vectors that you can use to test the correctness of your final code. Please, only submit your code if it is functional.
- In order to use the script, just put “`eea_gmp.c`” and the rest of the provided files along with the “`VerifyingEEA.sh`” script in one folder and run the following command in the terminal:
`bash VerifyingEEA.sh`
- The verification script uses the same format as the one that is run to grade your submissions, with different test vectors.
- You Must submit 1 file for this part of the assignment:
`eea_gmp.c`

Rubric:

- The “**Result.txt**” file [20 pt]

4 Programming Assignment 4: Pollard-Rho [45 pt]

In this exercise we will put into action the Pollard-Rho algorithm as described in “`Crash_NumberTheoryIntro.pptx`”, slide 35, to attack the discrete logarithm problem. Recall that the discrete logarithm problem is where you are given Y, α , and p from the equation $Y = \alpha^x \bmod p$, and you want to find x . Pollard-Rho achieves this in better-than-brute-force time complexity by exploiting the birthday paradox to find a collision in a sequence of group elements. Specifically, the algorithm uses a cycle-finding algorithm to compute a, b, A, B such that $\alpha^a Y^b = \alpha^A Y^B$, and since we can replace Y with α^x we can solve for x by solving the equivalent equation $(B - b)x = (A - a) \bmod p$.

Some online resources that may be helpful in creating or checking test cases involving modular exponentiation of large numbers:

<https://bigprimes.org/> for generating large prime numbers

<https://www.boxentriq.com/code-breaking/modular-exponentiation> for calculating the modular exponentiation of large numbers

<https://www.wolframalpha.com/widgets/view.jsp?id=ef51422db7db201ebc03c8800f41ba99> for finding the primitive roots of a large number (since α mentioned above must be a primitive root of p to ensure that there

exists only one solution.

Pollard-Rho:

1. The program reads the generator a , the result Y , and the modulo p from the files “**PRA.txt**”, “**PRY.txt**”, and “**PRP.txt**” respectively.
2. The program does Pollard-Rho’s algorithm with Floyd’s cycle-finding walk (also known as the tortoise and hare method, or the one-step two-step method) with randomized starting values.
3. If Floyd’s cycle-finding walk finds a valid cycle, in the sequence $x_i = \alpha_i^a Y_i^b$, then check if the collision is nontrivial and thus a valid solution to the logarithm problem.
4. If the walk fails to find a solution, repeat the algorithm with new randomized starting values.
5. When the program finds the solution, the program writes the found exponent in decimal string format to a file named “**Exponent.txt**”.

Programming Assignment 4 - Specific Notes

- Do not use library functions that solve for the exponent, perform modular exponentiation/inversion, or do the Extended Euclidean Algorithm directly.
You should re-use your implementations from Assignments 1-3, modifying them appropriately.
- Since we are working with up to 64-bit modulo, we will be using the `_int128` datatype (typedef’d to `int128_t` in `RequiredFunctionsHW2.c`). Some functions to work with `_int128` integers are provided in `RequiredFunctionsHW2.c`.
- Compile your code with the following commands:
gcc prattack.c -o pra
- Run your code with the first test files with the following command:
./pra PRA1.txt PRY1.txt PRP1.txt
- We have provided a script (`VerifyingPRACurve.sh`) and test vectors that you can use to test the correctness of your final code. Please, only submit your code if it is functional.
- This testing script and provided test vectors will also show the runtime of your code on a set of increasingly large modulus, in addition to an R-squared score of linear regression fit to a specific curve, giving you the information you need to answer the subsequent questions about Pollard-Rho.
- Since the Pollard-Rho algorithm attacking DLP has an element of randomness to it, the linear regression fit on runtimes will occasionally produce a low R-squared score even for correctly implemented code. The grader will run the Verification script multiple times in that case, minimizing the occurrence of false negatives.
- When creating test cases to test your code, make sure that p is prime and α is a generator (or primitive root) of p to ensure that the discrete logarithm problem has a solution (and only one solution). The provided test vectors follow this rule and thus a correctly implemented algorithm will always find a solution given enough runtime over enough randomized restarts.
- In order to use the script, just put “`prattack.c`” and the rest of the provided files along with the “`Verifying-PRACurve.sh`” script in one folder and run the following command in the terminal:
bash VerifyingPRACurve.sh
- The verification script uses the same format as the one that is run to grade your submissions, only with different test vectors.

- You Must submit 1 file for this part of the assignment:
[prattack.c](#)

Rubric:

- The “Result.txt” file [35 pt]
- Passes the grader’s verification and linear regression fit [10 pt]

Common Assignment Notes

- For the correctness of reading from a file, please use unsigned char.
- The files store the number as decimal strings, so, for example, reading “100” from a file gives you the string “100”, which represents the integer 100. Likewise the output of the results should also be in decimal string format, so a resulting integer of 99 should be written as the string “99” to the file.
- For your convenience, we have provided a template file with the necessary functions and descriptions: RequiredFunctionsHW2.c
- You may use the functions in RequiredFunctionsHW2.c, modify them, or write your own, modifying the RequiredFunctionsHW2.c file as necessary. **If you modify it, submit your modified RequiredFunctionsHW2.c file alongside your other code files in Canvas.**
- On Assignments 1, 2, and 3, it is recommended to encapsulate your algorithm logic into a function so that you can easily copy and reuse it in later assignments.
- The code that throws errors or does not successfully compile/run on our side, receives zero credit (It doesn’t matter if it works on your computer!).
- Follow the Instructions and all the specific names of output files mentioned above, otherwise there will be 20% deduction for inconvenience.

5 Question and Answer [5 pt + 10 pt (extra credit)]

5.1 Complexity of the Pollard-Rho algorithm [5 pt]

Given the runtimes you have experienced with running your Pollard-Rho code on test cases of varying sizes alongside your theoretical knowledge of Pollard-Rho’s algorithm, what is the time complexity of the Pollard-Rho algorithm and what variable is it based on? Write a paragraph explaining how you came to your answer.

5.2 Extra Credit: Shor’s algorithm [+10 pt Extra Credit]

- Describe in a paragraph or two the basis of Shor’s algorithm and how it relates to Pollard-Rho’s algorithm for attacking factorization or the discrete logarithm problem. [5 pt]
 - Describe in a paragraph or two: What makes Shor’s algorithm a quantum algorithm? How does it utilize its quantumness to attack the factorization problem/DLP? [5 pt]
- You must submit 1 file for this part of the assignment:
[HW2Group\(YourGroupNumber\).pdf](#)